

```

1  (*
2      CS 51 Lab 19
3      Reaction-Diffusion Cellular Automaton
4
5      *****
6      WARNING: If you have photosensitive epilepsy, you should avoid
7      viewing the reaction-diffusion cellular automaton. It exhibits
8      rapidly flashing visual patterns.
9      *****
10 *)
11
12 module G = Graphics ;;
13
14 (* Automaton parameters *)
15 let cGRID_SIZE = 100 ;;      (* width and height of grid in cells *)
16 let cSPARSITY = 5 ;;         (* inverse of proportion of cells initially live *)
17
18 (* Rendering parameters *)
19 let cCOLOR_LEGEND = G.rgb 173 106 108 ;; (* color for textual legend *)
20 let cSIDE = 8 ;;             (* width and height of cells in pixels *)
21 let cRENDER_FREQUENCY = 1    (* how frequently grid is rendered (in ticks) *) ;;
22 let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
23
24 type rd_state = float
25
26 (* offset index off -- Returns the `index` offset by `off` allowing
27    for wraparound. *)
28 let rec offset (index : int) (off : int) : int =
29     if index + off < 0 then
30         cGRID_SIZE - (offset ~-index ~-off)
31     else
32         (index + off) mod cGRID_SIZE ;;
33
34 (* rd_update grid i j -- Returns the updated value for cell at `i, j`
35    in the grid based on some simple reaction diffusion rules. *)
36 let rd_update (grid : rd_state array array) (i : int) (j : int) =
37     let neighbors = ref 0. in
38     for i' = ~-1 to ~+1 do
39         for j' = ~-1 to ~+1 do
40             let i_ind = offset i i' in
41             let j_ind = offset j j' in
42             neighbors := !neighbors +. grid.(i_ind).(j_ind)
43         done
44     done;
45     let norm = (!neighbors -. grid.(i).(j)) /. 8. in
46     let reacted = norm -. 4.
47         *. (norm -. 0.1)
48         *. (norm -. 0.5)
49         *. (norm -. 0.9) in
50     let clipped = min 1. (max 0. reacted) in
51     clipped ;;
52

```

```

53 (* rd_initialize grid -- Fills `grid` with uniform random values in [0, 1]. *)
54 let rd_initialize (grid : rd_state array array) : unit =
55     for i = 0 to cGRID_SIZE - 1 do
56         for j = 0 to cGRID_SIZE - 1 do
57             grid.(i).(j) <- Random.float 1.
58         done
59     done
60
61 module RDSpec : (Cellular.AUT_SPEC
62                 with type state = rd_state) =
63     struct
64         type state = rd_state
65         let name : string = "Turing's RD"
66         let initial : state = 0.1
67         let grid_size : int = cGRID_SIZE
68         let initialize = rd_initialize
69         let update = rd_update
70         let cell_color (cell : state) : G.color =
71             let col = (int_of_float (cell *. 255.)) in
72             if not ( 0 <= col && col < 256 ) then
73                 (Printf.printf "%f -> %d\n" cell col;
74                  raise (Invalid_argument "color out of range"))
75             else
76                 G.rgb col col col
77         let side_size : int = cSIDE
78         let legend_color : G.color = cCOLOR_LEGEND
79         let font : string option = cFONT
80         let render_frequency : int = cRENDER_FREQUENCY
81     end ;;
82
83 module Aut = Cellular.Automaton (RDSpec) ;;
84
85 (* .....
86    Running the automaton and displaying the results
87    *)
88
89 (* main seed -- Runs the automaton using the provided random `seed`
90    (for replicability), displaying updates to the graphics window. *)
91 let main seed =
92
93     Random.init seed;      (* initialize randomness *)
94     Aut.graphics_init ();  (* ... and graphics window *)
95
96     Aut.run_grid (Aut.initialized_grid ()) ;;
97
98 (* Run the automaton with user-supplied seed *)
99 let _ =
100     if Array.length Sys.argv <= 1 then
101         Printf.printf "Usage: %s <seed>\n where <seed> is integer seed\n"
102             Sys.argv.(0)
103     else
104         main (int_of_string Sys.argv.(1)) ;;
105

```